Checkpoint 5 — Synthetic Data Generator Implementation

Yassine — Amen Bank Internship

June 12, 2026

1 Overview

This checkpoint documents the implementation of the synthetic data generator, covering the project structure, class architecture, configuration system, amount calibration sources, and initial validation results. The generator produces a single CSV of labeled transaction events conforming to the locked raw event schema.

2 Project Structure

```
amen-anomaly/
 src/
    generator/
        __init__.py
        archetype.py
        client.py
        generator.py
        main.py
 config/
    archetypes.yaml
 data/
    transactions.csv
 models/
 docker/
 tests/
 .env
 README.md
```

3 Configuration System

Archetype parameters are externalized in `config/archetypes.yaml`, loaded via `pyyaml`. The YAML file contains:

- Population proportions (5 archetypes)
- Per-archetype Layer 1 parameters across 6 behavioral dimensions: operation mix, frequency (mean/std per operation), timing (day-of-month mean/std), amount (mean/std per operation), counterparty known ratio, branch loyalty

Separation of config from code allows parameter tuning without modifying Python source.

4 Amount Calibration Sources

Archetype amount parameters were calibrated using Tunisian salary data retrieved June 2026:

Metric	Value (DT/month)	Source
SMIG (minimum wage, 48h regime)	∼528	BCT / Al Qatiba 2025
Private sector average	∼665	CRLDHT 2026
National average (gross)	∼1,200–1,570	wage.is / employsome.com
National median	∼600	wage.is 2026
IT sector range	2,500–7,000+	employsome.com 2026

Key assumption: Salaried archetype represents banked individuals receiving salary deposits. Financial inclusion in Tunisia is ∼37% of adults, so bank clients skew above the national median. Central estimate for salaried archetype: ∼1,350 DT/month.

Derivation of retrait amount mean: Approximately 60–70% of salary withdrawn as cash (∼800–900 DT), divided across 3–4 monthly retraits ⇒ ∼250–300 DT per retrait. Config value: 300 DT.

Defensibility note: Exact values are calibrated estimates. Model performance depends on inter-archetype structural differences (cash flow inversion, frequency scaling, amount scaling), not absolute parameter precision. Parameters are externalized in YAML for iterative tuning.

5 Class Architecture

Class	Implementation Details
`Archetype`	Holds Layer 1 parameters loaded from YAML via `**kwargs` unpacking. One instance per archetype (5 total). `load_archetypes()` factory function returns a dict of archetypes and population proportions.
`Client`	Layer 2 personality sampling at creation. Gaussian draw from archetype parameters with constraints: `np.clip` for probabilities $\in [0, 1]$, non-negative frequencies, operation mix re-normalized to sum to 1. Personal amount std narrowed to 0.3× archetype std (progressive narrowing principle). Preferred day clipped to $[1, 30]$; edge cases handled at generation time via `min(preferred_day, last_day_of_month)`.
`Generator`	Two-phase execution. **Phase 1 (Planning):** creates client population per proportions, estimates total event volume, computes plan entries adjusted for duration (fix for anomaly rate inflation bug). **Phase 2 (Generation):** ticks day-by-day, computes daily transaction probability per client based on proximity to preferred day, generates normal or anomalous events based on active scenario windows.
`Event`	Inline in generator (no separate class needed). Flat envelope columns + JSON payload string. Labels (`is_anomaly`, `anomaly_type`) included for evaluation only, stripped before model training.

6 Anomaly Rate Bug Fix

Initial implementation assigned one plan entry per target anomalous event, ignoring that each entry's time window produces multiple anomalous events. Result: 1.52% anomaly rate vs. 0.13% target (∼10× inflation).

Fix: Divide target events by expected events per plan entry:

$$n_{\text{entries}} = \left\lfloor \frac{n_{\text{target events}}}{\text{duration} \times \bar{\lambda}_{\text{daily}}} \right\rfloor$$

where $\bar{\lambda}_{\text{daily}}$ is the average daily event rate per client. Post-fix anomaly rate: 0.147%.

7 Validation Results (10,000 clients, 180 days)

Metric	Value
Total events	342,759
Unique clients	9,948
Anomalies	504
Anomaly rate	0.147%

Operation	Count	Mean (DT)	Std (DT)	Max (DT)
Retrait	59,092	442.88	314.27	10,000
Versement	140,457	1,114.60	1,255.58	20,000
Virement	126,197	7,114.00	7,880.45	198,190
Chèque	17,013	5,620.59	8,057.53	41,972

All 13 anomaly scenarios represented in output. Difficulty distribution reflects window duration: hard scenarios (21-day windows) produce more events per plan entry than easy (1-day).

8 Decisions Logged

Decision	Rationale
D24: YAML config for archetype parameters	Decouples tuning from code; enables iterative calibration without modifying Python source.
D25: `pathlib`-based path resolution	`Path(__file__).resolve().parent.parent.parent` ensures config/data paths resolve correctly regardless of working directory.
D26: Preferred day clipped to $[1, 30]$	Avoids invalid dates; February edge case handled at generation time via `min(preferred_day, last_day_of_month)`.
D27: Anomaly plan entries adjusted for duration	Prevents anomaly rate inflation caused by multi-day windows generating multiple events per plan entry.

9 Known Gaps (Next Session)

- Several anomaly scenarios are stubs (e.g. `frequency_burst` is a pass-through)

- Employee assignment is random; should be branch-linked
- `depositor_id` for versements not populated
- Known beneficiary/emitter tracking uses placeholders (`KNOWN-1` etc.)
- Client count rounding loss (9,948 vs. 10,000 target)